Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Новгородский государственный университет имени Ярослава Мудрого»
(НовГУ)
Политехнический институт

Кафедра информационных технологий и систем

СЦЕНАРИИ ВЗАИМОДЕЙСТВИЯ И БИЗНЕС-ПРОЦЕССЫ программной системы «Контур»

Выполнил:
студент группы 2093
Граванов М.А.
Проверил:
руководитель ВКР / преподаватель
_____

Великий Новгород
2026 г.

Содержание

1 Диаграммы последовательности 3
1.1 Основной сценарий: создание задачи и фиксация связанного контекста 3
1.2 Альтернативный сценарий: отказ в изменении статуса задачи агентом 3
1.3 Сложный сценарий: синхронизация артефактов GitHub и файлового хранилища 4
2 BPMN-диаграмма процесса 5
2.1 Участники процесса 5
2.2 Структура процесса 5
Приложение А 7
Приложение Б 10

1 Диаграммы последовательности

Диаграммы последовательности используются для описания поведения системы во времени и показывают порядок обмена сообщениями между участниками сценария. Для системы «Контур» выбраны три ключевых сценария: основной пользовательский путь, альтернативный сценарий отказа в доступе и сложный интеграционный сценарий с внешними системами. Соответствующие диаграммы приведены в приложении А на рисунках А1, А2 и А3.

1.1 Основной сценарий: создание задачи и фиксация связанного контекста

В качестве основного сценария выбран процесс создания задачи в проекте. Этот путь является типовым для координатора или исполнителя и напрямую связан с центральной функцией системы — управлением работами. В сценарии участвуют веб-клиент, API-сервис, модуль проверки доступа, сервис управления задачами, модуль связывания сущностей, PostgreSQL и асинхронный обработчик аудита.

Сценарий начинается с заполнения формы задачи в пользовательском интерфейсе и отправки синхронного HTTP-запроса в API. После проверки сессии и прав доступа сервис задач создает запись в базе данных. Если при создании задачи выбраны связанные документы, заметки или файловые ресурсы, система переходит к условной ветви и в цикле добавляет соответствующие связи. После успешного завершения синхронной части формируется асинхронное сообщение в очередь событий, по которому отдельный обработчик сохраняет запись аудита. Такой подход позволяет сразу вернуть пользователю результат создания задачи и не задерживать ответ операциями фонового журналирования.

1.2 Альтернативный сценарий: отказ в изменении статуса задачи агентом

Альтернативный сценарий моделирует ситуацию, в которой агент пытается изменить статус задачи без необходимого разрешения на такую операцию. В этом сценарии взаимодействуют агент, MCP-мост, API-сервис, модуль

аутентификации и проверки набора разрешений, сервис задач и контур аудита. Диаграмма показывает точку возникновения ошибки, а также то, что бизнес-операция прерывается до момента изменения данных.

Запрос начинается с вызова инструмента агентом, после чего MCP-мост преобразует его в синхронный HTTP-запрос к API. Ошибка возникает на этапе проверки действительности ключа и набора разрешений. Если ключ не содержит нужного разрешения, запрос не передается в сервис изменения задачи, а вызывающая сторона получает ответ с кодом 403 Forbidden и сообщением об отказе в доступе. Для случая недействительного или истекшего ключа в диаграмме также может быть показана альтернативная ветвь с кодом 401 Unauthorized. Отказ фиксируется отдельным асинхронным событием аудита, что позволяет сохранить прослеживаемость неуспешных попыток изменения состояния задачи.

1.3 Сложный сценарий: синхронизация артефактов GitHub и файлового хранилища

Сложный сценарий описывает обработку задачи, связанной с внешними инженерными артефактами. Пользователь инициирует синхронизацию карточки задачи, после чего API-сервис создает асинхронное задание и передает его в очередь сообщений. Далее отдельный обработчик координирует взаимодействие как минимум с двумя внешними интеграциями: GitHub API и S3-совместимым файловым хранилищем.

Внутри сценария показано параллельное выполнение двух ветвей. В первой ветви обработчик получает по HTTP сведения о связанных issue, pull request, commit и branch из GitHub. Во второй ветви обработчик записывает в объектное хранилище сформированный снимок связанных файловых материалов и метаданные вложений. Для обеих ветвей предусматриваются таймауты и цикл повторных попыток. Если внешний вызов не завершается успешно за отведенное время, обработчик выполняет ограниченное число retry-повторов, после чего переводит задачу синхронизации в состояние частичного или полного отказа. После успешного завершения параллельных ветвей метаданные сохраняются в PostgreSQL, а в очередь публикуется событие для дальнейшей индексации и отражения результата в пользовательском интерфейсе.

2 BPMN-диаграмма процесса

В качестве ключевого бизнес-процесса для BPMN-моделирования выбран процесс обработки задачи от постановки до завершения. Такой выбор обусловлен тем, что именно задача является центральной сущностью системы «Контур», а ее жизненный цикл объединяет действия координатора, исполнителя и самой системы.

2.1 Участники процесса

В BPMN-диаграмме используются три дорожки: «Координатор», «Исполнитель» и «Система «Контур»». Координатор отвечает за постановку и приемку результата. Исполнитель выполняет практическую работу по задаче и передает итог на проверку. Система обеспечивает валидацию данных, контроль зависимостей, отправку уведомлений, отслеживание сроков и фиксацию событий аудита. Сама диаграмма приведена в приложении Б на рисунке Б1.

2.2 Структура процесса

Процесс начинается стартовым событием постановки новой задачи координатором. Далее выполняется последовательность действий и решений:

– координатор вводит атрибуты задачи и инициирует ее создание;

– система выполняет валидацию данных и через эксклюзивный шлюз выбирает одну из ветвей: [данные корректны] или [обнаружены ошибки заполнения];

– при корректных данных задача сохраняется, а система фиксирует событие создания в журнале аудита;

– затем через второй эксклюзивный шлюз определяется, есть ли у задачи блокирующие зависимости: [блокировок нет] или [задача заблокирована];

– при наличии блокировки запускается ожидание с промежуточным таймерным событием, после которого система повторно проверяет состояние зависимостей;

– если блокировки отсутствуют, процесс проходит через параллельный шлюз: в одной ветви система отправляет уведомление исполнителю, а в другой

начинает контроль срока выполнения и формирование напоминаний;

– исполнитель принимает задачу в работу, выполняет необходимые действия и передает результат на проверку через промежуточное событие сообщения;

– координатор рассматривает результат и через третий эксклюзивный шлюз принимает решение: [результат принят] или [требуется доработка];

– при необходимости доработки задача возвращается исполнителю и цикл выполнения повторяется;

– при успешной приемке система переводит задачу в завершенное состояние, записывает итоговое событие аудита и завершает процесс конечным событием.

В выбранном процессе отражены как штатные, так и исключительные ситуации. К исключениям относятся ошибки первичной валидации, ожидание снятия блокирующих зависимостей, просрочка срока выполнения с отправкой напоминания и возврат результата на доработку после проверки координатором. Такая структура позволяет на BPMN-диаграмме показать не только линейный маршрут выполнения задачи, но и реальные точки принятия решений, параллельную работу системы и возвраты по циклу управления.

Приложение А

В приложении А приведены диаграммы последовательности ключевых сценариев системы.

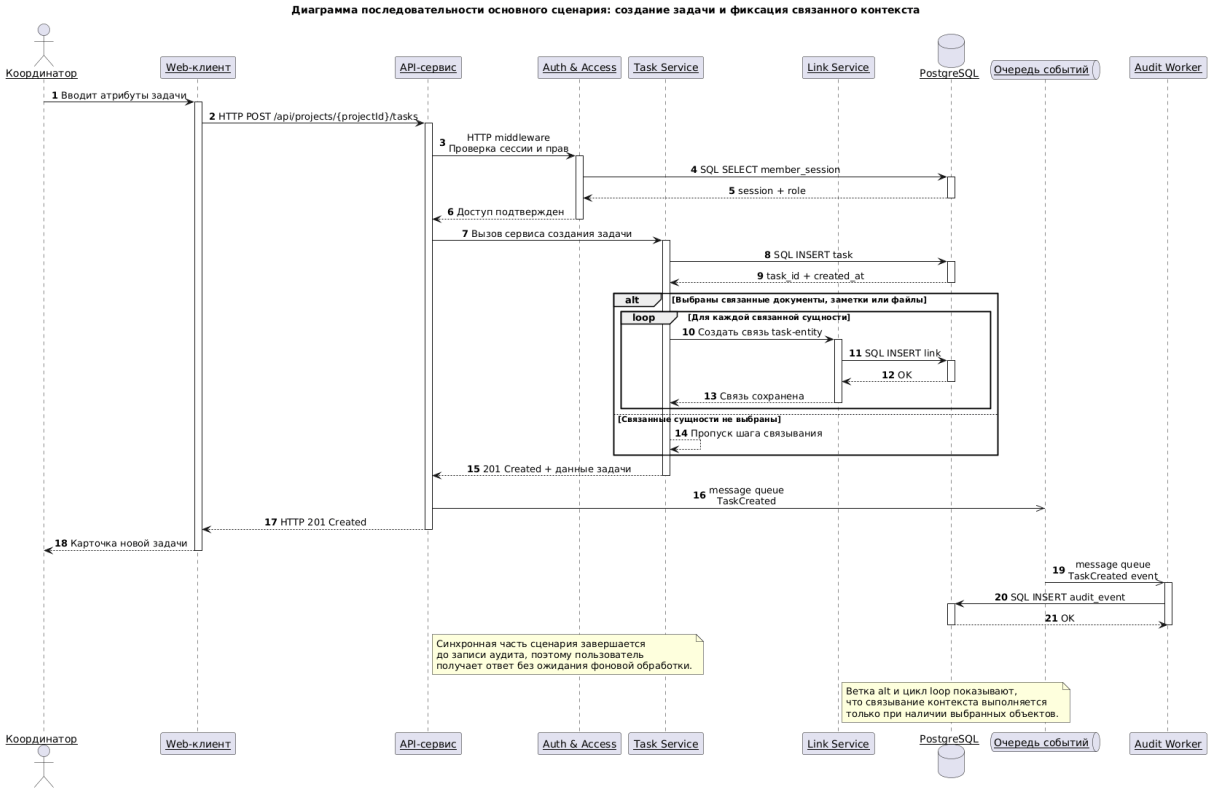

Рисунок А1 – Диаграмма последовательности основного сценария: создание задачи и фиксация связанного контекста

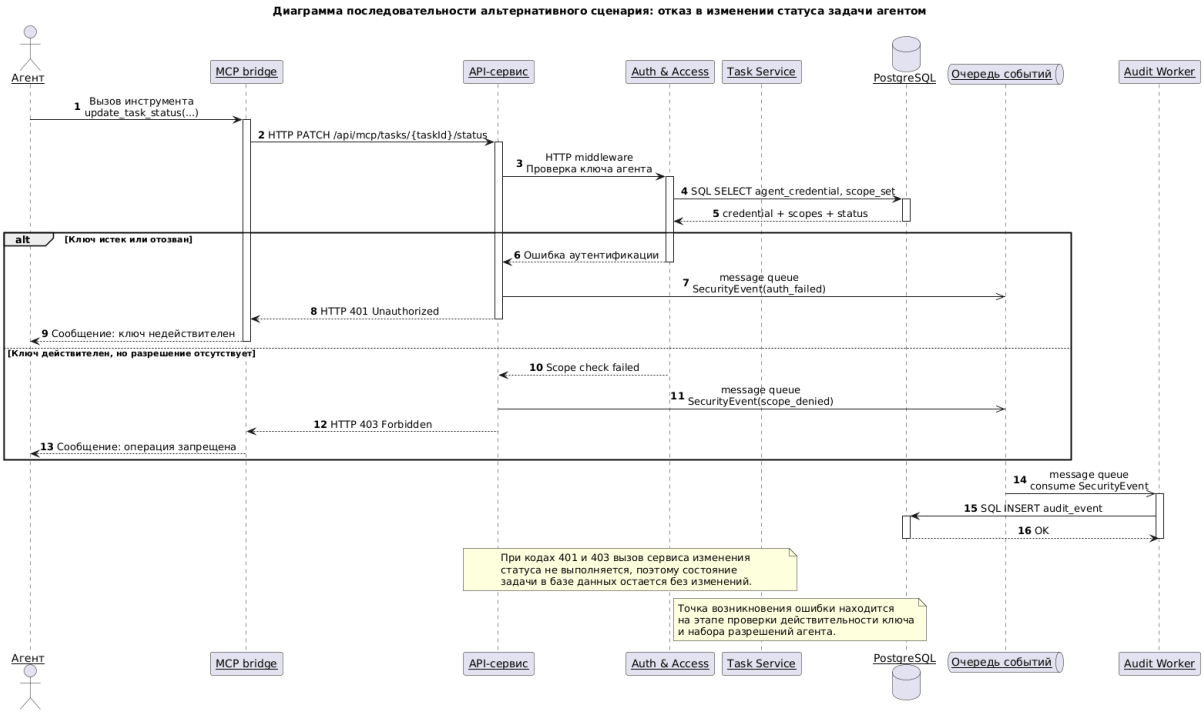

Рисунок А2 – Диаграмма последовательности альтернативного сценария: отказ в изменении статуса задачи агентом

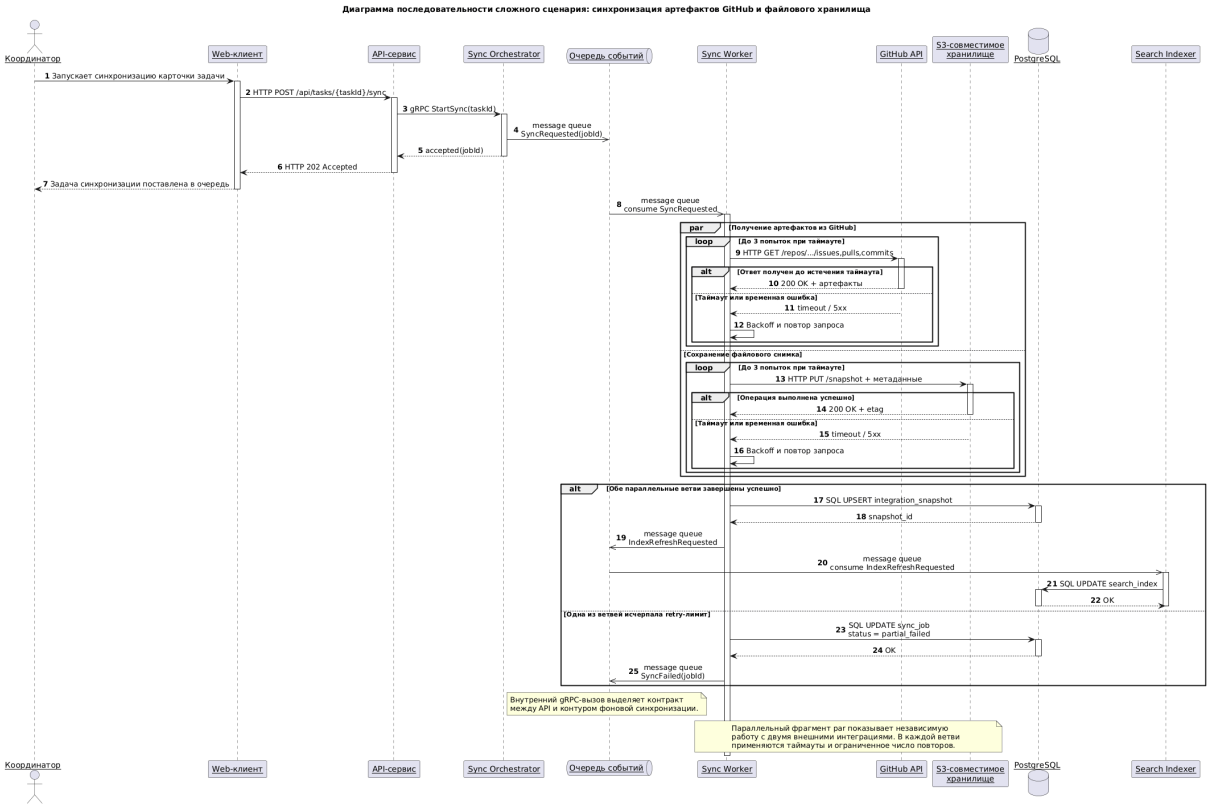

Рисунок А3 – Диаграмма последовательности сложного сценария: синхронизация артефактов GitHub и файлового хранилища

Приложение Б

В приложении Б приведена BPMN-диаграмма ключевого бизнес-процесса системы.

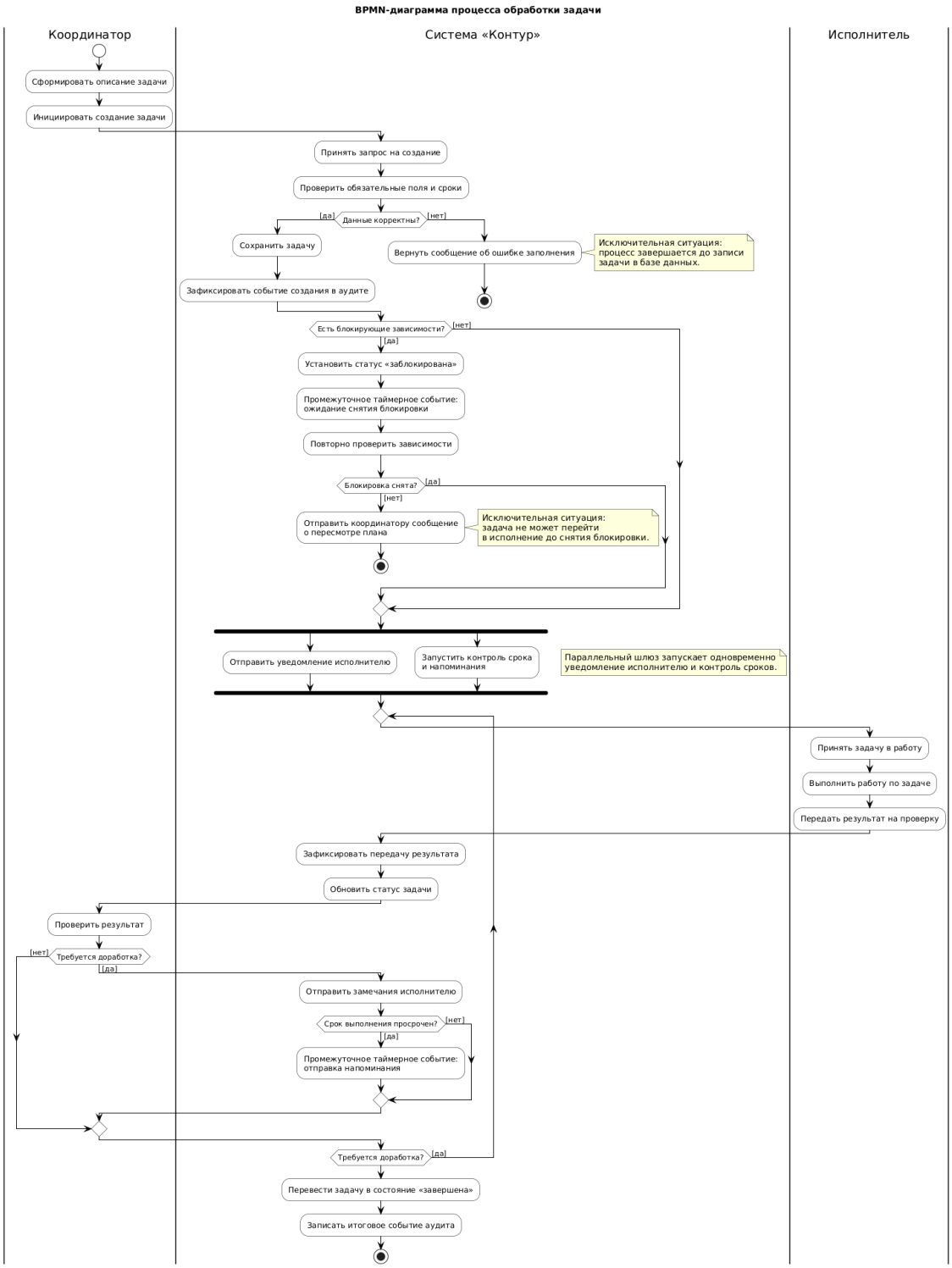

Рисунок Б1 – BPMN-диаграмма процесса обработки задачи